

WEB DEV-II QUESTION BANK - COMPLETE ANSWERS

DOM & JavaScript Basics

1. Define the DOM and explain its role in web development.

Answer:

The DOM (Document Object Model) is a programming interface that converts static HTML pages into a tree-like hierarchical structure that JavaScript can access and manipulate.

Key Characteristics:

- **Created by:** Server-side or browser
- **Purpose:** Connect JavaScript with HTML pages
- **Role:** Makes web pages dynamic and interactive by allowing runtime changes without page reload

Why DOM is Essential:

When we create a page using HTML, it is static and cannot respond to user actions. The DOM enables:

1. Dynamic content changes without reloading
2. Responding to user interactions (clicks, inputs, etc.)
3. Real-time modification of page content and styling
4. Making websites interactive and responsive

Example:

```
// Without DOM: Static HTML page, no interaction
// With DOM: Can dynamically change content
document.getElementById('heading').textContent = "New Content";
```

2. Explain DOM vs HTML

Aspect	HTML	DOM
Type	Markup language	Programming interface
Functionality	Adds content to webpage	Connects HTML with JavaScript
Interactivity	Static - doesn't respond to user actions	Dynamic - responds to user actions
Persistence	Cannot change after load	Can be changed at runtime
Purpose	Structure and presentation	Enable dynamic manipulation
Representation	Text-based code	Tree-like object structure

3. Difference between `textContent` , `innerHTML` , and `innerText`

Property	Description	Returns	Sets	Visibility
<code>textContent</code>	Gets/sets plain text content, ignoring HTML tags	Only text	Only text	Works even with hidden elements
<code>innerHTML</code>	Gets/sets HTML content including tags	HTML code + text	HTML code	Works with HTML tags
<code>innerText</code>	Gets/sets visible text only	Only visible text	Only visible text	Respects CSS visibility

Example:

```
<div id="box">
  <p>Hello <b>World</b></p>
</div>

const element = document.getElementById('box');

console.log(element.textContent); // "Hello World"
console.log(element.innerHTML);  // "<p>Hello <b>World</b></p>"
console.log(element.innerText);  // "Hello World"
```

4. Difference between `getElementById()` , `getElementsByClassName()` , `querySelector()` , `querySelectorAll()`

Method	Returns	Count	Details
getElementById()	Single element	1 element max	Selects by ID attribute only, returns null if not found
getElementsByClassName()	HTMLCollection	Multiple	Returns live collection, can use array methods like length and index
querySelector()	Single element	1 element	Selects first matching CSS selector, returns null if not found
querySelectorAll()	NodeList	Multiple	Returns static list, supports <code>forEach()</code> method

Key Differences:

- **getElementById()** - Returns only one element (IDs are unique)
- **getElementsByClassName()** - Returns HTMLCollection (array-like but not true array)
- **querySelector()** - Returns first element matching CSS selector
- **querySelectorAll()** - Returns NodeList, supports `forEach()`

Example:

```
// getElementById
let hero = document.getElementById('one');

// getElementsByClassName
let persons = document.getElementsByClassName('two');
console.log(persons[1]);    // indexed
console.log(persons.length); // length

// querySelector
let first = document.querySelector('#one'); // By ID
let first = document.querySelector('.two'); // By class
let first = document.querySelector('p');    // By tag

// querySelectorAll
let all = document.querySelectorAll('.hii');
```

Event Handling

5. What are classList methods (add/remove/toggle)?

Answer:

`classList` is a JavaScript property that provides methods to dynamically manage CSS classes on HTML elements.

Methods:

1. **add()** - Adds one or more classes to an element

```
element.classList.add("className");
element.classList.add("class1", "class2");
```

2. **remove()** - Removes one or more classes from an element

```
element.classList.remove("className");
element.classList.remove("class1", "class2");
```

3. **toggle()** - Adds class if not present, removes if already present

```
element.classList.toggle("className");  
// If class exists → remove it  
// If class doesn't exist → add it
```

4. **contains()** - Checks if element has a specific class

```
if (element.classList.contains("className")) {  
    // Class exists  
}
```

Example:

```
let head = document.getElementById('heading');  
  
head.classList.add("highlight");    // Add class  
head.classList.remove("highlight"); // Remove class  
head.classList.toggle("active");    // Toggle on/off  
head.classList.contains("active");  // Check if exists
```

6. Discuss event handling in JavaScript using `addEventListener()` with different event types.

Answer:

Event handling is how JavaScript responds to user actions like clicks, key presses, and mouse movements.

What is `addEventListener()` :

- Attaches an event listener to an HTML element
- Syntax: `element.addEventListener(eventType, function)`
- Makes web pages dynamic and interactive

Steps to Apply Event Listener:

1. Select the element
2. Define a function to execute
3. Use `addEventListener()` to attach the function

Example:

```
const button = document.getElementById('btn');

function handleClick() {
  alert("Button clicked!");
}

button.addEventListener("click", handleClick);
```

Different Event Types:

Mouse Events:

```
button.addEventListener("click", () => {
  console.log("Clicked");
});

button.addEventListener("mouseover", () => {
  button.style.backgroundColor = "yellow";
});

button.addEventListener("mouseout", () => {
  button.style.backgroundColor = "blue";
});
```

Keyboard Events:

```
document.addEventListener("keydown", (event) => {
  console.log("Key pressed:", event.key);
});

document.addEventListener("keyup", (event) => {
  console.log("Key released:", event.key);
});
```

Form Events:

```
const form = document.getElementById("myform");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // Prevent form reset
  console.log("Form submitted");
});
```

7. Explain event propagation. Differentiate between bubbling and capturing. Also discuss the role of event listeners.

Answer:

Event propagation describes how events travel through the DOM when an action occurs.

Three Phases of Event Propagation:

1. Capturing Phase (Top-Down)

- Event starts from document root
- Travels downward toward target element
- Example: document → html → body → div → button

2. Target Phase

- Event reaches the actual target element where action occurred

3. Bubbling Phase (Bottom-Up) - DEFAULT

- Event travels upward from target to parent elements
- Example: button → div → body → html → document

Visual Example:

Capturing: document → html → body → div → button

Target: button

Bubbling: button → div → body → html → document

Code Example:

```
let parent = document.getElementById("first");
let child = document.getElementById("button");

// Bubbling (default - false)
parent.addEventListener("click", () => {
    alert("Parent clicked (bubbling)");
}, false);

// Capturing (true)
parent.addEventListener("click", () => {
    alert("Parent clicked (capturing)");
}, true);

child.addEventListener("click", () => {
    alert("Child clicked");
});
```

Stopping Propagation:

```
element.addEventListener("click", (event) => {
    event.stopPropagation(); // Stops event from bubbling up
});
```

Role of Event Listeners:

- Detect user actions on specific elements
- Allow control over event flow
- Enable dynamic page responses
- Can be added/removed dynamically

8. Create a simple webpage using HTML, CSS, and JavaScript that demonstrates button click events and event removal.

Answer:


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Button Click Event Demo</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      background-color: #f0f0f0;
    }

    .container {
      text-align: center;
      background: white;
      padding: 40px;
      border-radius: 10px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }

    #text {
      font-size: 24px;
      color: black;
      margin: 20px 0;
    }

    button {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
      margin: 10px;
      border: none;
      border-radius: 5px;
      background-color: #007bff;
      color: white;
    }

    button:hover {
      background-color: #0056b3;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Button Click Event Demo</h1>
    <p>Click the button below to trigger an event.</p>
    <button>Click Me</button>
  </div>
</body>
</html>
```

```

    }
</style>
</head>
<body>
  <div class="container">
    <h1>Button Event Demo</h1>

    <h2 id="text">Welcome! Click the button below</h2>

    <button id="btn">Click Me</button>
  </div>

  <script>
    const button = document.getElementById('btn');
    const textElement = document.getElementById('text');

    // Define event handler function
    function handleClick() {
      textElement.textContent = "Button was clicked!";
      textElement.style.color = "green";

      // Remove event listener after first click
      button.removeEventListener("click", handleClick);
      button.textContent = "Event Removed";
      button.style.backgroundColor = "gray";
    }

    // Add event listener
    button.addEventListener("click", handleClick);
  </script>
</body>
</html>

```

Async JavaScript

9. Discuss microtask and macrotask queue.

Answer:

JavaScript's asynchronous tasks are managed using two priority queues:

Microtask Queue (High Priority):

- Executes immediately after current code
- Tasks: `.then()` , `.catch()` , `.finally()` , `Promise.resolve()`

Macrotask Queue (Low Priority):

- Executes after all microtasks complete
- Tasks: `setTimeout()` , `setInterval()` , `fetch()` , DOM events

Execution Order:

1. Call Stack (Synchronous code)
2. Microtask Queue (Promises)
3. Macrotask Queue (`setTimeout`, `fetch`)
4. Event Loop controls the flow

Example:

```
console.log("Start");           // 1. Sync - executes first

setTimeout(() => {
  console.log("Macrotask");      // 4. Macrotask - executes last
}, 0);

Promise.resolve().then(() => {
  console.log("Microtask");      // 3. Microtask - executes before macro
});

console.log("End");             // 2. Sync - executes second

// Output:
// Start
// End
// Microtask
// Macrotask
```

10. Difference between `setTimeout()` and `setInterval()`

Feature	<code>setTimeout()</code>	<code>setInterval()</code>
Purpose	Executes once after delay	Executes repeatedly at intervals
Syntax	<code>setTimeout(function, delay)</code>	<code>setInterval(function, interval)</code>
Execution	Runs once	Runs repeatedly
Stopping	Runs automatically after delay	Must use <code>clearInterval()</code> to stop
Use Case	Delayed single action	Repeated actions (timers, animations)

`setTimeout()` Example:

```
setTimeout(() => {  
    console.log("Executes after 3 seconds");  
}, 3000);
```

`setInterval()` Example:

```
let timer = setInterval(() => {  
    console.log("Repeats every 2 seconds");  
}, 2000);
```

```
// Stop after 10 seconds  
setTimeout(() => {  
    clearInterval(timer);  
    console.log("Timer stopped");  
}, 10000);
```

Important Point:

`setTimeout` only runs after:

1. The delay completes
2. The call stack becomes empty

```
setTimeout(() => {  
    console.log("A");  
}, 0);  
  
console.log("B");  
  
// Output:  
// B  
// A  
// (B executes first even though setTimeout has 0 delay)
```

11. What is Callback Hell? Explain its disadvantages.

Answer:

Callback Hell (also called "Pyramid of Doom") occurs when callbacks are nested deeply within each other.

Example of Callback Hell:

```

function boilWater(callback) {
  setTimeout(() => {
    console.log("Water boiled");
    callback();
  }, 2000);
}

function addMaggie(callback) {
  setTimeout(() => {
    console.log("Maggie added");
    callback();
  }, 2000);
}

function addMasala(callback) {
  setTimeout(() => {
    console.log("Masala added");
    callback();
  }, 2000);
}

// Callback Hell - deeply nested
boilWater(function() {
  addMaggie(function() {
    addMasala(function() {
      console.log("Ready to eat!");
    });
  });
});

```

Disadvantages of Callback Hell:

1. **Hard to Maintain** - Code becomes difficult to understand and maintain as program grows
2. **Error Handling Difficult** - Handling errors in nested callbacks is complicated
3. **Deeply Nested Code** - Makes code messy and hard to read
4. **Inversion of Control** - When passing callbacks, we lose control over when/how they execute
5. **Debugging Complexity** - Stack traces become hard to follow
6. **Code Readability** - Pyramid shape makes code hard to follow

Solutions:

1. Use Promises with `.then()` chaining

2. Use `async/await` syntax
3. Use named functions instead of inline callbacks

12. What are Promise states in JavaScript?

Answer:

A Promise represents an asynchronous operation that can be in one of three states:

Three States:

1. Pending

- Initial state
- Operation is still running
- Not settled yet

2. Fulfilled (Resolved)

- Operation completed successfully
- `resolve()` was called
- Result is available

3. Rejected

- Operation failed
- `reject()` was called
- Error information is available

State Transitions:

Pending → Fulfilled (resolved with result)
→ Rejected (rejected with error)

Once a Promise changes state, it CANNOT change again!

Code Example:

```

let promise = new Promise((resolve, reject) => {
  let success = true;

  if (success) {
    resolve("Task completed"); // Pending → Fulfilled
  } else {
    reject("Task failed");      // Pending → Rejected
  }
});

promise
  .then(result => console.log(result)) // Handles resolved state
  .catch(error => console.log(error))  // Handles rejected state
  .finally(() => console.log("Done")); // Runs in both cases

```

13. Explain the Event Loop and how asynchronous code executes in JavaScript.

Answer:

JavaScript is a single-threaded language. The Event Loop manages asynchronous operations using multiple components:

Four Main Components:

1. Call Stack

- Where all JavaScript code executes
- Executes line by line
- Keeps track of function calls

2. Web API

- Provided by browser
- Handles time-consuming tasks (setTimeout, fetch, localStorage)
- Not part of JavaScript

3. Task Queue (Callback Queue)

- Holds completed asynchronous tasks

- Two sub-queues:
 - Microtask Queue (high priority) - Promises
 - Macrotask Queue (low priority) - setTimeout, fetch

4. Event Loop

- Manager that checks if call stack is empty
- If empty, moves task from queue to call stack
- Ensures correct execution order

Execution Flow:

1. Synchronous code → Call Stack (executes immediately)
2. Async code (setTimeout, fetch) → Web API (waits)
3. Web API completes → Task Queue (waits in line)
4. Event Loop checks Call Stack
5. If Call Stack empty → Move task to Call Stack
6. Task executes

Visual Example:

```
console.log("Start");           // Step 1: Call Stack

setTimeout(() => {
  console.log("Timeout");       // Step 6: Execute after all sync code
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");       // Step 5: Microtask (before macrotask)
});

console.log("End");             // Step 2: Call Stack

// Output:
// Start
// End
// Promise
// Timeout
```

14. Explain execution context in JavaScript including GEC, FEC, and call stack.

Answer:

Execution context is the environment where JavaScript code is executed.

Types of Execution Context:

1. Global Execution Context (GEC)

- Created when JavaScript starts running
- Available globally to all code
- Only one GEC per program

2. Function Execution Context (FEC)

- Created when a function is called
- Each function call creates new FEC
- Multiple FEC can exist

Call Stack:

A data structure that stores execution contexts and manages their execution order using LIFO (Last In First Out) principle.

How It Works:

```
function first() {  
    second();  
}  
  
function second() {  
    third();  
}  
  
function third() {  
    console.log("Hello");  
}  
  
first(); // Call Stack: first() → second() → third()
```

Call Stack Visualization:

Step 1: first() called

└─ Call Stack: [Global, first()]

Step 2: first() calls second()

└─ Call Stack: [Global, first(), second()]

Step 3: second() calls third()

└─ Call Stack: [Global, first(), second(), third()]

Step 4: third() executes console.log

└─ Output: "Hello"

Step 5: third() completes and pops

└─ Call Stack: [Global, first(), second()]

Step 6: second() completes and pops

└─ Call Stack: [Global, first()]

Step 7: first() completes and pops

└─ Call Stack: [Global]

Why Call Stack is Important:

1. Manages function execution order
2. Coordinates with Event Loop and Callback Queue
3. Helps in debugging - shows which functions are active
4. Ensures correct variable scope

15. Explain the lifecycle of a Promise

Answer:

Problem of Callback Hell:

When multiple callbacks are nested, code becomes:

- Hard to read and maintain
- Difficult to debug
- Prone to errors
- Inversion of control

Definition of Promise:

A Promise is an object that represents the eventual result of an asynchronous operation that will finish in the future.

States (Lifecycle):

1. **Pending** - Initial state, operation in progress
2. **Fulfilled** - Operation succeeded, `resolve()` called
3. **Rejected** - Operation failed, `reject()` called

Consumers:

- `.then()` - Runs when promise resolves successfully
- `.catch()` - Runs if any error occurs
- `.finally()` - Runs always, regardless of state

Implementation Example:

```

function boilWater() {
  return new Promise((resolve) => {
    console.log("Water added to pot");
    setTimeout(() => {
      console.log("Water boiled");
      resolve();
    }, 2000);
  });
}

function addMaggie() {
  return new Promise((resolve) => {
    console.log("Adding maggie");
    setTimeout(() => {
      console.log("Maggie added");
      resolve();
    }, 2000);
  });
}

function addMasala() {
  return new Promise((resolve, reject) => {
    console.log("Adding masala");
    let masalaAvailable = false;

    setTimeout(() => {
      if (masalaAvailable) {
        console.log("Masala added");
        resolve();
      } else {
        reject("Masala not available");
      }
    }, 2000);
  });
}

// Promise Chaining
boilWater()
  .then(() => addMaggie())
  .then(() => addMasala())
  .then(() => console.log("Serve and eat"))

```

```
.catch((error) => console.log("Error:", error))  
.finally(() => console.log("Cooking process finished"));
```

Advantages Over Callbacks:

-  Cleaner code
-  Easier to read
-  Easier to debug
-  Better error handling

16. Explain `async/await` with error handling using `try...catch`

Answer:

`async/await` is modern syntax that makes asynchronous code look and behave like synchronous code.

`async` Keyword:

- Makes a function always return a Promise
- Allows use of `await` inside the function

```
async function greet() {  
    return "Hello";  
}
```

```
// Returns: Promise { 'Hello' }
```

```
greet().then(msg => console.log(msg)); // Output: Hello
```

`await` Keyword:

- Pauses function execution until Promise settles
- Can only be used inside `async` function
- Returns the resolved value

Example Without Error Handling:

```
function boilWater() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log("Water boiled");  
      resolve();  
    }, 2000);  
  });  
}
```

```
function addMaggie() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log("Maggie added");  
      resolve();  
    }, 2000);  
  });  
}
```

```
async function cookMaggi() {  
  await boilWater();  
  await addMaggie();  
  console.log("Ready to eat!");  
}
```

```
cookMaggi();
```

With Error Handling (try...catch):

```
async function getData() {  
  try {  
    let response = await fetch("https://api.example.com/users");  
  
    if (!response.ok) {  
      throw new Error("HTTP Error: " + response.status);  
    }  
  
    let data = await response.json();  
    console.log(data);  
  }  
  catch (error) {  
    console.log("Error occurred:", error.message);  
  }  
  finally {  
    console.log("Request completed");  
  }  
}  
  
getData();
```

Key Points:

- try block: Contains code that might throw error
- catch block: Handles errors if they occur
- finally block: Runs regardless of success/failure

Advantages of async/await:

-  Easier to read
-  Easier to write
-  Easier to debug
-  Better error handling
-  Avoids callback hell

Storage

17. Differentiate between Local Storage and Session Storage

Feature	Local Storage	Session Storage
Persistence	Data persists even after browser closes	Data cleared when tab/browser closes
Scope	Global - accessible from all tabs	Tab-specific - not shared between tabs
Size Limit	5-6 MB	5-6 MB
Data Format	Only stores strings	Only stores strings
Expiration	Manually removed only	Auto-removes on tab close
Use Cases	Website theme, user preferences	Form data, temporary shopping cart
Auto-Cleanup	No - requires manual clearing	Yes - on page/tab close

Local Storage Methods:

```
// Set item
localStorage.setItem("name", "Sameer");

// Get item
let name = localStorage.getItem("name");

// Remove item
localStorage.removeItem("name");

// Clear all
localStorage.clear();

// Storing objects (must convert to JSON)
let user = { name: "Sameer", age: 19 };
localStorage.setItem("user", JSON.stringify(user));

// Retrieving objects (must parse from JSON)
let userData = JSON.parse(localStorage.getItem("user"));
```

Session Storage Methods:

```
// Same methods as localStorage
sessionStorage.setItem("tempData", "value");
let data = sessionStorage.getItem("tempData");
sessionStorage.removeItem("tempData");
sessionStorage.clear();
```

Example - Remembering Theme:

```
// Store theme
function setTheme(theme) {
  localStorage.setItem("theme", theme);
}

// Retrieve theme on page load
function getTheme() {
  return localStorage.getItem("theme") || "light";
}

// Apply saved theme
window.addEventListener("load", () => {
  let savedTheme = getTheme();
  document.body.classList.add(savedTheme);
});
```

React Basics

18. What is JSX in React?

Answer:

JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows you to write HTML-like code inside JavaScript.

Key Characteristics:

- Looks like HTML but is actually JavaScript
- Must be compiled by Babel to regular JavaScript
- Makes React components easier to read and write
- Each JSX element becomes a React element

JSX Syntax:

```
const element = <h1>Hello, World!</h1>;

const name = "Sameer";
const greeting = <h1>Hello, {name}!</h1>;

const component = (
  <div>
    <h1>My App</h1>
    <p>Welcome to React</p>
  </div>
);
```

How JSX is Compiled:

```
// JSX
const element = <h1 className="greeting">Hello</h1>;

// Compiled to JavaScript
const element = React.createElement(
  'h1',
  { className: 'greeting' },
  'Hello'
);
```

Rules of JSX:

1. Single root element required
2. HTML attributes use camelCase (className, onClick, onChange)
3. CSS properties use camelCase (backgroundColor, fontSize)
4. Expressions wrapped in curly braces {}
5. JSX elements are self-closing or closed properly

Example:

```
function App() {  
  const name = "Sameer";  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      <h1>Welcome, {name}!</h1>  
      {isLoggedIn ? <p>You are logged in</p> : <p>Please log in</p>}  
      <button onClick={() => alert("Clicked!")}>Click Me</button>  
    </div>  
  );  
}
```

19. What is one-way data binding?

Answer:

One-way data binding means data flows in only one direction: from parent component to child component.

How It Works:

- Parent component passes data to child component via props
- Child component receives data but cannot modify parent state directly
- Changes in parent automatically update child
- Child cannot send data back to parent without callback

Advantages:

- ☒ Easier to debug (data flow is clear)
- ☒ More predictable
- ☒ Better performance
- ☒ Easier to maintain

Example:

```

// Parent Component
function Parent() {
  const [message, setMessage] = React.useState("Hello Child");

  return (
    <div>
      <Child data={message} />
      <button onClick={() => setMessage("Updated")}>
        Update
      </button>
    </div>
  );
}

// Child Component
function Child({ data }) {
  return <p>Received: {data}</p>;
  // Child cannot modify 'data' directly
}

```

20. Define props in React with a simple use case

Answer:

Props (short for properties) are read-only data passed from parent components to child components. They allow components to receive and use dynamic values.

Characteristics:

- Read-only - child cannot modify props
- Used for component communication
- Passed as attributes in JSX
- Accessed via function parameters

Simple Use Case: User Card Component

```
// Parent Component
function App() {
  return (
    <div>
      <UserCard name="Sameer" age={19} city="Delhi" />
      <UserCard name="John" age={25} city="Mumbai" />
      <UserCard name="Sarah" age={22} city="Bangalore" />
    </div>
  );
}

// Child Component
function UserCard({ name, age, city }) {
  return (
    <div style={{ border: "1px solid black", padding: "10px" }}>
      <h3>Name: {name}</h3>
      <p>Age: {age}</p>
      <p>City: {city}</p>
    </div>
  );
}
```

How Props Work:

1. Parent passes data as attributes: `<UserCard name="Sameer" age={19} />`
2. Child receives via function parameters: `function UserCard({ name, age })`
3. Child uses the data: `<h3>{name}</h3>`
4. When parent updates data, child automatically updates

Hooks, State & Props

21. What are hooks? State the rules of hooks

Answer:

Hooks are special functions that let you "hook into" React features like state and lifecycle methods in functional components.

Common Hooks:

- `useState()` - Manage component state
- `useEffect()` - Handle side effects
- `useContext()` - Access context
- `useReducer()` - Complex state management

Example - `useState` Hook:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Rules of Hooks:

1. Only Call at Top Level

- Don't call hooks inside loops, conditions, or nested functions
- Must be called at the top of functional components

```
// ✅ Correct
function Component() {
  const [count, setCount] = useState(0); // Top level
  // ...
}

// ❌ Wrong
function Component() {
  if (condition) {
    const [count, setCount] = useState(0); // Inside condition
  }
}
```

2. Only Call from React Functions

- Call hooks from functional components only
- Can call from custom hooks
- Don't call from regular JavaScript functions

```
//  Correct - In React component
function MyComponent() {
  const [data, setData] = useState(null);
}

//  Wrong - In regular function
function regularFunction() {
  const [data, setData] = useState(null);
}
```

3. Custom Hooks Pattern

- Custom hooks can call other hooks
- Must start with "use" prefix
- Can reuse logic across components

22. Demonstrate the role of `useState` using a simple example

Answer:

`useState` is a hook that lets functional components have state. It returns current state value and function to update it.

Syntax:

```
const [state, setState] = useState(initialValue);
```

Simple Counter Example:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ padding: "20px", textAlign: "center" }}>
      <h1>Counter Application</h1>
      <h2>Current Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrement
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}

export default Counter;
```

Form Input Example:

```

import React, { useState } from 'react';

function FormDemo() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Name:", name);
    console.log("Email:", email);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="email"
        placeholder="Enter email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <button type="submit">Submit</button>
    </form>
  );
}

export default FormDemo;

```

Key Points:

- count - Current state value
- setCount - Function to update state
- Updating state triggers re-render
- useState can be called multiple times

23. Explain the use of dependency array in `useEffect`

Answer:

The dependency array controls when a `useEffect` hook should run.

`useEffect` Syntax:

```
useEffect(() => {  
    // Side effect code  
}, [dependencies]);
```

Dependency Array Cases:

1. No Dependency Array - Runs Every Render

```
useEffect(() => {  
    console.log("Runs on every render");  
});
```

2. Empty Dependency Array - Runs Once (on mount)

```
useEffect(() => {  
    console.log("Runs once when component mounts");  
}, []);
```

3. With Dependencies - Runs When Dependencies Change

```
useEffect(() => {  
    console.log("Runs when 'count' or 'name' changes");  
}, [count, name]);
```

Practical Example:

```

import React, { useState, useEffect } from 'react';

function UserProfile() {
  const [userId, setUserId] = useState(1);
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(false);

  // Fetch user when userId changes
  useEffect(() => {
    setLoading(true);

    fetch(`https://api.example.com/users/${userId}`)
      .then(response => response.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      });
  }, [userId]); // Runs when userId changes

  return (
    <div>
      <h1>User Profile</h1>

      <button onClick={() => setUserId(userId + 1)}>
        Next User
      </button>

      {loading ? <p>Loading...</p> : <p>{user?.name}</p>}
    </div>
  );
}

export default UserProfile;

```

Cleanup Function:

```

useEffect(() => {
  // Setup
  console.log("Setup");

  // Cleanup (runs before effect runs again or component unmounts)
  return () => {
    console.log("Cleanup");
  };
}, [dependency]);

```

24. Disadvantage of props drilling

Answer:

Props drilling (also called prop threading) is passing props through multiple intermediate components to reach a deeply nested component that actually needs the data.

Problem Illustration:

```

// App.js - Has user data
function App() {
  const user = { name: "Sameer", age: 19 };
  return <Level1 user={user} />; // Pass user
}

// Level1.js - Doesn't need user, just passes it
function Level1({ user }) {
  return <Level2 user={user} />; // Pass user
}

// Level2.js - Doesn't need user, just passes it
function Level2({ user }) {
  return <Level3 user={user} />; // Pass user
}

// Level3.js - Finally uses user
function Level3({ user }) {
  return <h1>Welcome {user.name}</h1>;
}

```

Disadvantages:

1. Code Maintenance Difficult

- Tracking props through multiple components is hard
- Difficult to refactor

2. Performance Issues

- Unnecessary re-renders of intermediate components
- Even if component doesn't use prop, it re-renders

3. Code Clutter

- Intermediate components cluttered with props they don't use
- Reduces readability

4. Error Prone

- Easy to forget passing prop at intermediate level
- Typos can break entire chain

5. Scalability Issues

- As app grows, becomes unmanageable
- Multiple levels of drilling causes complexity

Example of Issues:

```
// If we modify any part of this chain, entire app might break
function App() {
  const user = { name: "Sameer" };
  const theme = "dark";
  const settings = { language: "en" };

  // All passed through drilling
  return <Component1 user={user} theme={theme} settings={settings} />;
}
```

Routing & Context API

25. Explain Context API and compare it with props drilling. Also implement data sharing across nested components

Answer:

Context API provides a way to pass data through component tree without having to pass props down manually at every level.

Why Context API:

- Avoids props drilling
- Centralized data management
- Cleaner component code
- Easier to maintain

How Context API Works:

Step 1: Create Context

```
const UserContext = React.createContext();
```

Step 2: Create Provider

```
function UserProvider({ children }) {  
  const [user, setUser] = useState({ name: "Sameer", age: 19 });  
  
  return (  
    <UserContext.Provider value={{ user, setUser }}>  
      {children}  
    </UserContext.Provider>  
  );  
}
```

Step 3: Use Context

```
function UserProfile() {  
  const { user } = React.useContext(UserContext);  
  return <h1>Welcome {user.name}</h1>;  
}
```

Complete Example:


```
import React, { useState, useContext, createContext } from 'react';
```

```
// Create context
```

```
const ThemeContext = createContext();
```

```
// Create provider
```

```
function ThemeProvider({ children }) {  
  const [theme, setTheme] = useState("light");  
  
  const toggleTheme = () => {  
    setTheme(theme === "light" ? "dark" : "light");  
  };  
  
  return (  
    <ThemeContext.Provider value={{ theme, toggleTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}
```

```
// Component that uses context
```

```
function Header() {  
  const { theme, toggleTheme } = useContext(ThemeContext);  
  
  return (  
    <div style={{  
      background: theme === "light" ? "white" : "black",  
      color: theme === "light" ? "black" : "white",  
      padding: "20px"  
    }}>  
      <h1>My App</h1>  
      <button onClick={toggleTheme}>Toggle Theme</button>  
    </div>  
  );  
}
```

```
function Content() {  
  const { theme } = useContext(ThemeContext);  
  
  return (  
    <div style={{  
      background: theme === "light" ? "white" : "black",  
      color: theme === "light" ? "black" : "white",
```

```

        padding: "20px"
      }}>
      <p>Content area</p>
    </div>
  );
}

// App component
function App() {
  return (
    <ThemeProvider>
      <Header />
      <Content />
    </ThemeProvider>
  );
}

export default App;

```

Comparison: Props Drilling vs Context API

Aspect	Props Drilling	Context API
Data Path	Prop → Intermediate → Component	Direct access from any component
Code Complexity	Increases with depth	Constant regardless of depth
Intermediate Components	Cluttered with unused props	Clean, no unused props
Performance	Unnecessary re-renders	More controlled re-renders
Maintenance	Difficult to modify	Easy to modify
Use Case	1-2 level components	Deep nesting
Setup Time	None	Requires Context setup

Advantages of Context API:

-  Eliminates prop drilling
-  Cleaner component code
-  Easier to maintain

-  Better performance in deep trees

26. Discuss disadvantages of props drilling and how Context API solves them

Answer:

Disadvantages of Props Drilling:

1. Maintenance Nightmare

- Changes in prop structure require updating every intermediate component
- Easy to introduce bugs
- Difficult to track prop flow

// Before: If we rename 'user' prop, must update all levels

```
function App() {  
  return <Level1 user={user} />; // Change here  
}
```

```
function Level1({ user }) {  
  return <Level2 user={user} />; // Change here  
}
```

```
function Level2({ user }) {  
  return <Level3 user={user} />; // Change here  
}
```

2. Performance Issues

- Intermediate components re-render even if they don't use props
- Creates unnecessary render cycles
- Can slow down large apps

// Level1 and Level2 re-render just to pass props

```
function Level1({ user, theme, settings }) {  
  // Doesn't use these props, just passes them  
  return <Level2 user={user} theme={theme} settings={settings} />;  
}
```

3. Code Readability

- Components cluttered with unused props
- Hard to understand purpose of each prop
- Confusing for new developers

4. Error Proneness

- Easy to forget passing prop
- Typos in prop names break app
- Hard to debug issues

```
// Forgetting to pass one prop breaks entire chain
function Level2({ user, theme }) {
  return <Level3 user={user} />; // Forgot 'theme'!
}
```

How Context API Solves These:

Solution 1: Direct Access

```
// Any component can directly access data
function Level3() {
  const { user } = useContext(UserContext);
  return <h1>{user.name}</h1>;
}

// Level1 and Level2 don't need to pass props
function Level2() {
  return <Level3 />; // Clean!
}
```

Solution 2: Centralized Management

```
// All data in one place
const UserContext = createContext();

function UserProvider({ children }) {
  const [user, setUser] = useState({ name: "Sameer" });
  const [theme, setTheme] = useState("light");
  const [settings, setSettings] = useState({});

  return (
    <UserContext.Provider value={{ user, theme, settings }}>
      {children}
    </UserContext.Provider>
  );
}

// Any component accesses what it needs
function Component() {
  const { user } = useContext(UserContext);
  const { theme } = useContext(UserContext);
}
```

Solution 3: Easy Maintenance

- Change data structure in one place
- All components automatically get updates
- No need to modify intermediate components

Solution 4: Better Performance

- Only components that use context re-render
- Intermediate components stay clean
- Granular updates possible

Comparison Table:

Problem	Props Drilling	Context API
Tracking data flow	Hard	Easy
Number of prop edits	Many levels	One place
Intermediate component clutter	High	None

Problem	Props Drilling	Context API
Performance	Unnecessary re-renders	Optimized
Debugging	Difficult	Clear
Scalability	Poor	Excellent

27. Explain component lifecycle methods in React and how they are implemented using `useEffect` with examples

Answer:

Component lifecycle refers to the stages a component goes through: mount, update, and unmount.

Lifecycle Phases:

1. **Mounting** - Component is being created
2. **Updating** - Component is being re-rendered
3. **Unmounting** - Component is being removed

`useEffect` Implementation:

1. On Mount (`componentDidMount` equivalent)

```
useEffect(() => {  
  console.log("Component mounted");  
  // Fetch data, setup listeners, etc.  
}, []); // Empty dependency array = runs once on mount
```

2. On Update (`componentDidUpdate` equivalent)

```
useEffect(() => {  
  console.log("Component updated or dependency changed");  
  // Run when specific dependency changes  
}, [dependency]);
```

3. On Unmount (`componentWillUnmount` equivalent)

```
useEffect(() => {  
  // Setup  
  console.log("Component mounted");  
  
  // Cleanup function  
  return () => {  
    console.log("Component unmounting");  
    // Cleanup: remove listeners, cancel requests, etc.  
  };  
}, []);
```

Practical Example:

```

import React, { useState, useEffect } from 'react';

function UserData() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // On Mount - Fetch data
  useEffect(() => {
    console.log("Mounting - Fetching user data");

    fetch("https://api.example.com/user")
      .then(res => res.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err);
        setLoading(false);
      });

    // Cleanup on unmount
    return () => {
      console.log("Unmounting - Cleaning up");
    };
  }, []); // Only on mount

  // On Update - When user changes
  useEffect(() => {
    console.log("User data updated:", user);
  }, [user]); // When user changes

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return <h1>Welcome {user?.name}</h1>;
}

export default UserData;

```

Comparison: Class Component vs Functional Component

Lifecycle	Class Component	Functional Component
Mount	<code>componentDidMount()</code>	<code>useEffect(() => {...}, [])</code>
Update	<code>componentDidUpdate()</code>	<code>useEffect(() => {...}, [dep])</code>
Unmount	<code>componentWillUnmount()</code>	<code>useEffect(() => () => {...})</code>
Multiple effects	Combined	Separate hooks

API & Forms

28. Write the code– how to fetch API In React

Answer:

Fetching API data in React involves making HTTP requests to get data from a server.

Basic GET Request:

```

import React, { useState, useEffect } from 'react';

function FetchData() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(response => response.json())
      .then(data => {
        setData(data);
        setLoading(false);
      })
      .catch(error => {
        setError(error);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <div>
      <h1>Users List</h1>
      {data?.map(user => (
        <div key={user.id}>
          <h3>{user.username}</h3>
          <p>Email: {user.email}</p>
        </div>
      ))}
    </div>
  );
}

export default FetchData;

```

Complete Example: Fetch Users from JSONPlaceholder

```

import React, { useState, useEffect } from 'react';

function UsersList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchUsers = async () => {
      try {
        setLoading(true);

        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );

        if (!response.ok) {
          throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data = await response.json();
        setUsers(data);
        setError(null);
      }
      catch (error) {
        setError(error.message);
      }
      finally {
        setLoading(false);
      }
    };

    fetchUsers();
  }, []);

  if (loading) return <div className="loading">Loading users...</div>;
  if (error) return <div className="error">Error: {error}</div>;

  return (
    <div className="users-container">
      <h1>Users List</h1>
      <table border="1">
        <thead>

```

```

        <tr>
          <th>ID</th>
          <th>Username</th>
          <th>Email</th>
          <th>Phone</th>
        </tr>
      </thead>
      <tbody>
        {users.map(user => (
          <tr key={user.id}>
            <td>{user.id}</td>
            <td>{user.username}</td>
            <td>{user.email}</td>
            <td>{user.phone}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

```

```
export default UsersList;
```

With async/await:

```

useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch("https://api.example.com/data");
      const jsonData = await response.json();
      setData(jsonData);
    } catch (error) {
      setError(error);
    }
  };

  fetchData();
}, []);

```

POST Request Example:

```

async function createUser() {
  try {
    const response = await fetch(
      'https://jsonplaceholder.typicode.com/users',
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          name: 'John Doe',
          email: 'john@example.com',
          username: 'johndoe'
        })
      }
    );

    const data = await response.json();
    console.log('Created user:', data);
  } catch (error) {
    console.error('Error:', error);
  }
}

```

29. Design and implement a User Information Form using React

Answer:

```
import React, { useState } from 'react';
import './UserForm.css';

function UserInformationForm() {
  const [formData, setFormData] = useState({
    name: '',
    age: '',
    gender: '',
    languages: []
  });

  const [submittedData, setSubmittedData] = useState(null);
  const [submitted, setSubmitted] = useState(false);

  // Handle text and number inputs
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  };

  // Handle checkbox changes for languages
  const handleLanguageChange = (e) => {
    const { value, checked } = e.target;

    if (checked) {
      setFormData(prev => ({
        ...prev,
        languages: [...prev.languages, value]
      }));
    } else {
      setFormData(prev => ({
        ...prev,
        languages: prev.languages.filter(lang => lang !== value)
      }));
    }
  };

  // Handle form submission
  const handleSubmit = (e) => {
    e.preventDefault();
```

```

// Validation
if (!formData.name || !formData.age || !formData.gender) {
  alert('Please fill all required fields');
  return;
}

// Store submitted data
setSubmittedData(formData);
setSubmitted(true);

// Reset form after 3 seconds
setTimeout(() => {
  setFormData({
    name: '',
    age: '',
    gender: '',
    languages: []
  });
  setSubmitted(false);
}, 3000);
};

// Handle reset
const handleReset = () => {
  setFormData({
    name: '',
    age: '',
    gender: '',
    languages: []
  });
  setSubmitted(false);
  setSubmittedData(null);
};

return (
  <div className="form-container">
    <h1>User Information Form</h1>

    {!submitted ? (
      <form onSubmit={handleSubmit} className="form">
        {/* Name Field */}
        <div className="form-group">

```

```
<label htmlFor="name">Name (Text Input):</label>
<input
  type="text"
  id="name"
  name="name"
  placeholder="Enter your name"
  value={formData.name}
  onChange={handleInputChange}
  required
/>
</div>
```

```
{/* Age Field */}
<div className="form-group">
  <label htmlFor="age">Age (Number Input):</label>
  <input
    type="number"
    id="age"
    name="age"
    placeholder="Enter your age"
    value={formData.age}
    onChange={handleInputChange}
    min="1"
    max="120"
    required
  />
</div>
```

```
{/* Gender Select Box */}
<div className="form-group">
  <label htmlFor="gender">Gender (Select Box):</label>
  <select
    id="gender"
    name="gender"
    value={formData.gender}
    onChange={handleInputChange}
    required
  >
    <option value="">Select Gender</option>
    <option value="Male">Male</option>
    <option value="Female">Female</option>
    <option value="Other">Other</option>
  </select>
```



```

</div>

{/* Languages Checkbox */}
<div className="form-group">
  <label>Languages (Checkbox):</label>
  <div className="checkbox-group">
    {[ 'English', 'Spanish', 'French', 'Hindi' ].map(lang => (
      <label key={lang} className="checkbox-label">
        <input
          type="checkbox"
          value={lang}
          checked={formData.languages.includes(lang)}
          onChange={handleLanguageChange}
        />
        {lang}
      </label>
    ))}
  </div>
</div>

{/* Buttons */}
<div className="button-group">
  <button type="submit" className="btn-submit">
    Submit
  </button>
  <button type="reset" onClick={handleReset} className="btn-reset">
    Reset
  </button>
</div>
</form>
) : (
  <div className="submitted-data">
    <h2>Submitted Information</h2>
    <p><strong>Name:</strong> {submittedData.name}</p>
    <p><strong>Age:</strong> {submittedData.age}</p>
    <p><strong>Gender:</strong> {submittedData.gender}</p>
    <p><strong>Languages:</strong> {submittedData.languages.join(', ')}</p>
    <p className="success-message">Form submitted successfully!</p>
  </div>
)}
</div>
);
}

```

```
export default UserInformationForm;
```

CSS (UserForm.css):

```
.form-container {
    max-width: 500px;
    margin: 50px auto;
    padding: 30px;
    border: 2px solid #007bff;
    border-radius: 10px;
    background-color: #f9f9f9;
}

.form-container h1 {
    color: #333;
    text-align: center;
    margin-bottom: 30px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
    color: #333;
}

.form-group input,
.form-group select {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 5px;
    font-size: 14px;
    box-sizing: border-box;
}

.form-group input:focus,
.form-group select:focus {
    outline: none;
    border-color: #007bff;
    box-shadow: 0 0 5px rgba(0, 123, 255, 0.5);
}
```

```
.checkbox-group {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.checkbox-label {
  display: flex;
  align-items: center;
  gap: 8px;
  cursor: pointer;
}

.checkbox-label input {
  width: auto;
  cursor: pointer;
}

.button-group {
  display: flex;
  gap: 10px;
  margin-top: 20px;
}

.btn-submit,
.btn-reset {
  flex: 1;
  padding: 12px;
  border: none;
  border-radius: 5px;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
}

.btn-submit {
  background-color: #28a745;
  color: white;
}

.btn-submit:hover {
  background-color: #218838;
}
```

```
.btn-reset {  
  background-color: #dc3545;  
  color: white;  
}  
  
.btn-reset:hover {  
  background-color: #c82333;  
}  
  
.submitted-data {  
  background-color: #d4edda;  
  border: 1px solid #c3e6cb;  
  border-radius: 5px;  
  padding: 20px;  
  color: #155724;  
}  
  
.submitted-data h2 {  
  color: #155724;  
  margin-bottom: 15px;  
}  
  
.success-message {  
  color: #28a745;  
  font-weight: bold;  
  margin-top: 15px;  
}
```

30. Create a React component with two buttons and a counter state

Answer:

```

import React, { useState, useEffect } from 'react';

function CounterWithPopup() {
  const [counter, setCounter] = useState(0);
  const [showPopup, setShowPopup] = useState(false);

  // Control popup display
  useEffect(() => {
    if (showPopup) {
      // Auto-hide popup after 2 seconds
      const timeout = setTimeout(() => {
        setShowPopup(false);
      }, 2000);

      return () => clearTimeout(timeout);
    }
  }, [showPopup]);

  // First button - Shows popup every time clicked
  const handlePopupButton = () => {
    alert("Button 1 clicked! Counter value: " + counter);
    setShowPopup(true);
  };

  // Second button - Updates counter without popup
  const handleCounterButton = () => {
    setCounter(counter + 1);
  };

  return (
    <div style={styles.container}>
      <h1>Counter App with Popup Control</h1>

      <div style={styles.counterDisplay}>
        <h2>Current Count: {counter}</h2>
      </div>

      <div style={styles.buttonGroup}>
        {/* Button 1 - Shows popup */}
        <button
          onClick={handlePopupButton}
          style={styles.button1}
        >

```

```

        Show Popup (Counter: {counter})
    </button>

    {/* Button 2 - Updates counter silently */}
    <button
        onClick={handleCounterButton}
        style={styles.button2}
    >
        Update Counter
    </button>
</div>

{/* Popup */}
{showPopup && (
    <div style={styles.popup}>
        <div style={styles.popupContent}>
            <p>Button 1 was clicked!</p>
            <p>Counter is now: {counter}</p>
            <button
                onClick={() => setShowPopup(false)}
                style={styles.closeBtn}
            >
                Close
            </button>
        </div>
    </div>
    )}
</div>
);
}

```

```

const styles = {
    container: {
        textAlign: 'center',
        padding: '40px',
        fontFamily: 'Arial, sans-serif',
        maxWidth: '500px',
        margin: '50px auto',
        backgroundColor: '#f0f0f0',
        borderRadius: '10px',
    },
    counterDisplay: {
        backgroundColor: '#e3f2fd',
    },

```

```
padding: '30px',
borderRadius: '8px',
marginBottom: '30px',
},
buttonGroup: {
display: 'flex',
gap: '15px',
justifyContent: 'center',
flexWrap: 'wrap',
},
button1: {
padding: '15px 25px',
fontSize: '16px',
backgroundColor: '#ff9800',
color: 'white',
border: 'none',
borderRadius: '5px',
cursor: 'pointer',
fontWeight: 'bold',
},
button2: {
padding: '15px 25px',
fontSize: '16px',
backgroundColor: '#4caf50',
color: 'white',
border: 'none',
borderRadius: '5px',
cursor: 'pointer',
fontWeight: 'bold',
},
popup: {
position: 'fixed',
top: '0',
left: '0',
right: '0',
bottom: '0',
backgroundColor: 'rgba(0, 0, 0, 0.5)',
display: 'flex',
justifyContent: 'center',
alignItems: 'center',
zIndex: '1000',
},
popupContent: {
```



```
        backgroundColor: 'white',
        padding: '30px',
        borderRadius: '10px',
        boxShadow: '0 4px 6px rgba(0, 0, 0, 0.1)',
        textAlign: 'center',
    },
    closeBtn: {
        padding: '10px 20px',
        backgroundColor: '#2196f3',
        color: 'white',
        border: 'none',
        borderRadius: '5px',
        cursor: 'pointer',
        marginTop: '15px',
    },
};

export default CounterWithPopup;
```

31. Create a React functional component using `useState` to implement a counter application (`increment` , `decrement` , `reset`)

Answer:

```

import React, { useState } from 'react';
import './Counter.css';

function CounterApp() {
  const [count, setCount] = useState(0);
  const [history, setHistory] = useState([0]);

  // Increment counter
  const handleIncrement = () => {
    const newCount = count + 1;
    setCount(newCount);
    setHistory([...history, newCount]);
  };

  // Decrement counter
  const handleDecrement = () => {
    const newCount = count - 1;
    setCount(newCount);
    setHistory([...history, newCount]);
  };

  // Reset counter
  const handleReset = () => {
    setCount(0);
    setHistory([0]);
  };

  return (
    <div className="counter-container">
      <div className="counter-card">
        <h1>Counter Application</h1>

        <div className="counter-display">
          <h2 className="counter-value">{count}</h2>
        </div>

        <div className="button-group">
          <button
            onClick={handleDecrement}
            className="btn btn-decrement"
          >
            — Decrement
          </button>

```

```

        <button
          onClick={handleReset}
          className="btn btn-reset"
        >
          🔄 Reset
        </button>

        <button
          onClick={handleIncrement}
          className="btn btn-increment"
        >
          + Increment
        </button>
      </div>

      <div className="info-box">
        <p>Total Operations: {history.length - 1}</p>
        <p>Status: {count > 0 ? '📈 Positive' : count < 0 ? '📉 Negative' : '⚖️ Ne
      </div>

      <div className="history">
        <h3>History:</h3>
        <div className="history-list">
          {history.map((value, index) => (
            <span key={index} className="history-item">
              {value}
            </span>
          ))}
        </div>
      </div>
    </div>
  </div>
);
}

export default CounterApp;

```

CSS (Counter.css):

```
.counter-container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  padding: 20px;  
}
```

```
.counter-card {  
  background: white;  
  border-radius: 20px;  
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);  
  padding: 40px;  
  max-width: 500px;  
  width: 100%;  
}
```

```
.counter-card h1 {  
  text-align: center;  
  color: #333;  
  margin-bottom: 30px;  
  font-size: 28px;  
}
```

```
.counter-display {  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  border-radius: 15px;  
  padding: 40px;  
  text-align: center;  
  margin-bottom: 30px;  
  box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);  
}
```

```
.counter-value {  
  font-size: 72px;  
  color: white;  
  margin: 0;  
  font-weight: bold;  
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);  
}
```

```
.button-group {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 15px;
  margin-bottom: 30px;
}

.button-group .btn-reset {
  grid-column: 1 / -1;
}

.btn {
  padding: 15px 25px;
  border: none;
  border-radius: 10px;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.3s ease;
  color: white;
  text-transform: uppercase;
  letter-spacing: 1px;
}

.btn-increment {
  background-color: #4caf50;
}

.btn-increment:hover {
  background-color: #45a049;
  transform: scale(1.05);
  box-shadow: 0 5px 15px rgba(76, 175, 80, 0.4);
}

.btn-decrement {
  background-color: #ff9800;
}

.btn-decrement:hover {
  background-color: #e68900;
  transform: scale(1.05);
  box-shadow: 0 5px 15px rgba(255, 152, 0, 0.4);
}
```

```
.btn-reset {  
    background-color: #f44336;  
}  
  
.btn-reset:hover {  
    background-color: #da190b;  
    transform: scale(1.05);  
    box-shadow: 0 5px 15px rgba(244, 67, 54, 0.4);  
}  
  
.btn:active {  
    transform: scale(0.98);  
}  
  
.info-box {  
    background-color: #f5f5f5;  
    border-radius: 10px;  
    padding: 20px;  
    margin-bottom: 20px;  
    text-align: center;  
}  
  
.info-box p {  
    margin: 10px 0;  
    color: #333;  
    font-weight: 500;  
}  
  
.history {  
    background-color: #f0f7ff;  
    border-radius: 10px;  
    padding: 20px;  
    border-left: 4px solid #667eea;  
}  
  
.history h3 {  
    margin-top: 0;  
    color: #667eea;  
}  
  
.history-list {  
    display: flex;
```

```
    flex-wrap: wrap;
    gap: 10px;
}

.history-item {
    background-color: white;
    border: 2px solid #667eea;
    border-radius: 8px;
    padding: 8px 12px;
    font-weight: bold;
    color: #667eea;
    font-size: 14px;
}
```

Output Based Questions

32. Guess the output with reason of the output

Question:

```
console.log("Start");

async function test() {
    console.log("Inside function");

    await Promise.resolve();

    console.log("After await");
}

test();

setTimeout(() => {
    console.log("Timeout");
}, 0);

console.log("End");
```

Answer:

Output:

```
Start
Inside function
End
After await
Timeout
```

Explanation:

Execution Flow:

- Step 1: `console.log("Start")`**
 - Synchronous code executes immediately
 - **Output:** Start
 - Call Stack: [Global, console.log]
- Step 2: `test()` function called**
 - Function execution starts
 - **Output:** Inside function
 - Call Stack: [Global, test]
 - `await Promise.resolve()` - suspends function
 - Promise moved to Microtask Queue
- Step 3: `setTimeout()` called**
 - Sent to Web API
 - Timer starts (0ms delay)
 - Callback moved to Macrotask Queue
- Step 4: `console.log("End")`**
 - Synchronous code executes
 - **Output:** End
 - Call Stack becomes empty
- Step 5: Event Loop check - Microtask Queue first**
 - `await Promise.resolve()` completes
 - Function resumes
 - **Output:** After await
 - Call Stack: [Global, test]
- Step 6: Event Loop check - Macrotask Queue**
 - `setTimeout` callback executes
 - **Output:** Timeout
 - Call Stack: [Global, setTimeout]

Key Concepts:

Priority Order:

1. Call Stack (Synchronous code)
2. Microtask Queue (Promises, async/await)
3. Macrotask Queue (setTimeout, setInterval)

Call Stack State at Each Phase:

Initial: [Global]
After test(): [Global, test]
After await: [Global] ← Suspended, waiting for promise
After Promise resolves: [Global, test] ← Resumes
After setTimeout: [Global, console.log("Timeout")]
Final: [Global] ← Empty

Why This Order:

- **"Start" and "End"**: Synchronous code executes first
- **"Inside function"**: Function call starts synchronously
- **"After await"**: Promise resolves after all sync code, before setTimeout
- **"Timeout"**: setTimeout is lowest priority, executes last

Summary Answer:

The execution follows the JavaScript Event Loop mechanism:

1. All synchronous code runs first (Start, Inside function, End)
2. Microtasks (Promises) execute next (After await)
3. Macrotasks (setTimeout) execute last (Timeout)

This demonstrates how JavaScript prioritizes different types of asynchronous operations despite setTimeout having 0ms delay.

End of Complete Answers

Note: All answers are structured for exam preparation with:

-  Clear definitions and explanations
-  Practical code examples
-  Comparison tables where applicable
-  Visual diagrams and flow charts
-  Important points highlighted
-  Common pitfalls and best practices